

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра «Програмна інженерія»
Звіт з лабораторної роботи №2
З дисципліни "Генеративні нейронні мережі та їх застосування"
Використання та fine-tuning великих мовних моделей

Виконав:
студент гр. ПЗм-25-2
Вовк Дмитро Андрійович

Перевірила: доцент кафедри ПІ
Шевченко Олена Леонідівна

Харків 2026

Мета: Навчитися реалізовувати та використовувати базову DDPM на основі бібліотеки PyTorch

Хід роботи:

Візьмемо статтю aibhav Singh An In-Depth Guide to Denoising Diffusion Probabilistic Models DDPM – Theory to Implementation, 2023 р. (URL: <https://learnopencv.com/denoising-diffusion-probabilistic-models/>), як основу лабораторної роботи.

Як приклад коду до статті, використаємо: <https://github.com/gctian/DDPMs-from-scratch>

Змінемо базовий сет на «MNIST»

Configurations

```
from dataclasses import dataclass

@dataclass
class BaseConfig:
    DEVICE = get_default_device()
    DATASET = "MNIST" # "MNIST", "Cifar-10", "Cifar-100", "Flowers"

    # For logging inference images and saving checkpoints.
    root_log_dir = os.path.join("Logs_Checkpoints", "Inference")
    root_checkpoint_dir = os.path.join("Logs_Checkpoints", "checkpoints")

    # Current log and checkpoint directory.
    log_dir = "version_0"
    checkpoint_dir = "version_0"
```

image size:32×32

```
@dataclass
class TrainingConfig:
    TIMESTEPS = 1000 # Define number of diffusion timesteps
    IMG_SHAPE = (1, 32, 32) if BaseConfig.DATASET == "MNIST" else (3, 32, 32)
    NUM_EPOCHS = 30
    BATCH_SIZE = 128
    LR = 2e-4
    NUM_WORKERS = 2
```

зробити 100 timesteps на training та стільки ж на sampling:

```
@dataclass
class TrainingConfig:
    TIMESTEPS = 100 # Define number of diffusion timesteps
    IMG_SHAPE = (1, 32, 32) if BaseConfig.DATASET == "MNIST" else (3, 32, 32)
    NUM_EPOCHS = 30
    BATCH_SIZE = 128
    LR = 2e-4
    NUM_WORKERS = 2
```

```
def get_betas(self):
    """linear schedule, proposed in original ddpm paper"""
    scale = 100 / self.num_diffusion_timesteps
    beta_start = scale * 1e-4
    beta_end = scale * 0.02
    return torch.linspace(
        beta_start,
        beta_end,
```

```
[10] ▶ x0s, _ = next(loader)

noisy_images = []
specific_timesteps = [0, 1, 5, 10, 15, 20, 25, 30, 40, 60, 80, 99]

for timestep in specific_timesteps:
    timestep = torch.as_tensor(timestep, dtype=torch.long)

    xts, _ = forward_diffusion(sd, x0s, timestep)
    xts = inverse_transform(xts) / 255.0
```

та оновимо використання функції `reverse_diffusion` — змінимо значення для аргумента `timesteps`.

залишити 32 базових канали:

```
▶ @dataclass
class ModelConfig:
    BASE_CH = 32 # 64, 128, 256, 512
    BASE_CH_MULT = (1, 2, 4, 8) # 32, 16, 8, 4
    APPLY_ATTENTION = (False, False, True, False)
    DROPOUT_RATE = 0.1
    TIME_EMB_MULT = 2 # 128
```

```

class UNet(nn.Module):
    def __init__(
        self,
        input_channels=3,
        output_channels=3,
        num_res_blocks=2,
        base_channels=32,
        base_channels_multiples=(1, 2, 4, 8),
        dropout_rate=0.1,
        time_multiple=4,
    ):
        super().__init__()

        time_emb_dims_exp = base_channels * time_multiple
        self.time_embeddings = SinusoidalPositionEmbeddings(time_emb_dims=base_channels, time_emb_dims_exp=time_emb_dims_exp)

        self.first = nn.Conv2d(in_channels=input_channels, out_channels=base_channels, kernel_size=3, stride=1, padding="same")

        num_resolutions = len(base_channels_multiples)

        # Encoder part of the UNet. Dimension reduction.
        self.encoder_blocks = nn.ModuleList()
        curr_channels = [base_channels]
        in_channels = base_channels

        for level in range(num_resolutions):
            out_channels = base_channels * base_channels_multiples[level]

            for _ in range(num_res_blocks):
                block = ResnetBlock(
                    in_channels=in_channels,
                    out_channels=out_channels,
                    dropout_rate=dropout_rate,
                    time_emb_dims=time_emb_dims_exp,
                )
                self.encoder_blocks.append(block)

                in_channels = out_channels
                curr_channels.append(in_channels)

            if level != (num_resolutions - 1):
                self.encoder_blocks.append(DownSample(channels=in_channels))
                curr_channels.append(in_channels)

        # Bottleneck in between
        self.bottleneck_blocks = nn.ModuleList(
            (
                ResnetBlock(
                    in_channels=in_channels,
                    out_channels=in_channels,
                    dropout_rate=dropout_rate,
                    time_emb_dims=time_emb_dims_exp,
                ),
                ResnetBlock(
                    in_channels=in_channels,

```

```

    )
)

# Decoder part of the UNet. Dimension restoration with skip-connections.
self.decoder_blocks = nn.ModuleList()

for level in reversed(range(num_resolutions)):
    out_channels = base_channels * base_channels_multiples[level]

    for _ in range(num_res_blocks + 1):
        encoder_in_channels = curr_channels.pop()
        block = ResnetBlock(
            in_channels=encoder_in_channels + in_channels,
            out_channels=out_channels,
            dropout_rate=dropout_rate,
            time_emb_dims=time_emb_dims_exp
        )

        in_channels = out_channels
        self.decoder_blocks.append(block)

    if level != 0:
        self.decoder_blocks.append(UpSample(in_channels))

self.final = nn.Sequential(
    nn.GroupNorm(num_groups=8, num_channels=in_channels),
    nn.SiLU(),
    nn.Conv2d(in_channels=in_channels, out_channels=output_channels, kernel_size=3, stride=1, padding="same"),
)

def forward(self, x, t):

    time_emb = self.time_embeddings(t)

    h = self.first(x)
    outs = [h]

    for layer in self.encoder_blocks:
        h = layer(h, time_emb)
        outs.append(h)

    for layer in self.bottleneck_blocks:
        h = layer(h, time_emb)

    for layer in self.decoder_blocks:
        if isinstance(layer, ResnetBlock):
            out = outs.pop()
            h = torch.cat([h, out], dim=1)
            h = layer(h, time_emb)

    h = self.final(h)

    return h

```

зробити кількість рівнів UNet 3

```

@dataclass
class ModelConfig:
    BASE_CH = 32 # 64, 128, 256, 512
    BASE_CH_MULT = (1, 2, 4) # 32, 16, 8, 4
    DROPOUT_RATE = 0.1
    TIME_EMB_MULT = 2 # 128

```

1 residual-блок на рівень

```
class UNet(nn.Module):
    def __init__(
        self,
        input_channels=3,
        output_channels=3,
        num_res_blocks=1,
        base_channels=32,
        base_channels_multiple=1,
        dropout_rate=0.1,
        time_multiple=4,
    ):
        super().__init__()
```

5-10 епох навчання (5)

```
@dataclass
class TrainingConfig:
    TIMESTEPS = 100 # Define number of diffusion timesteps
    IMG_SHAPE = (1, 32, 32) if BaseConfig.DATASET == "MNIST" else (3, 32, 32)
    NUM_EPOCHS = 5
    BATCH_SIZE = 128
    LR = 2e-4
    NUM_WORKERS = 2
```

Графік loss

```
plt.plot(losses)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss")
plt.show()
```



Додати візуалізацію forward diffusion

```
x0s, _ = next(loader)

noisy_images = []
specific_timesteps = [0, 1, 5, 10, 15, 20, 25, 30, 40, 60, 80, 99]

for timestep in specific_timesteps:
    timestep = torch.as_tensor(timestep, dtype=torch.long)

    xts, _ = forward_diffusion(sd, x0s, timestep)
    xts = inverse_transform(xts) / 255.0
    xts = make_grid(xts, nrow=1, padding=1)

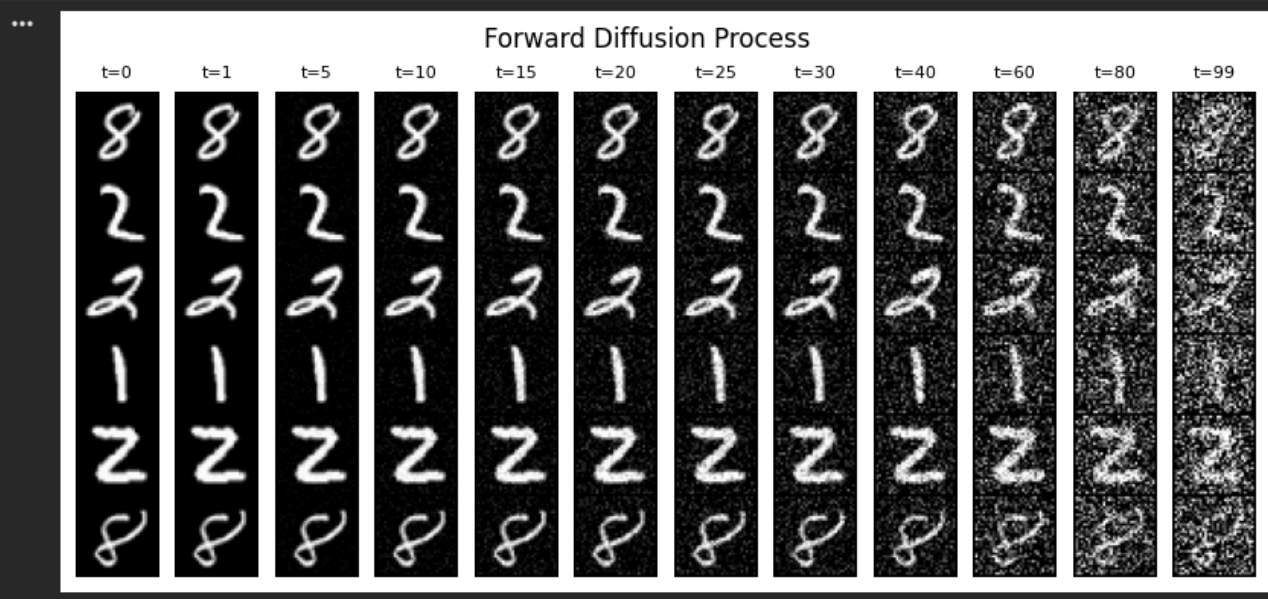
    noisy_images.append(xts)

# Plot and see samples at different timesteps

_, ax = plt.subplots(1, len(noisy_images), figsize=(10, 5), facecolor='white')

for i, (timestep, noisy_sample) in enumerate(zip(specific_timesteps, noisy_images)):
    ax[i].imshow(noisy_sample.squeeze(0).permute(1, 2, 0))
    ax[i].set_title(f"t={timestep}", fontsize=8)
    ax[i].axis("off")
    ax[i].grid(False)

plt.suptitle("Forward Diffusion Process", y=0.9)
plt.axis("off")
plt.show()
```



Додати візуалізацію reverse diffusion. Оновимо reverse_diffusion

```
# Algorithm 2: Sampling

@torch.inference_mode()
```

```

def reverse_diffusion(model, sd, timesteps=100, img_shape=(3, 32, 32),
                      num_images=5, nrow=8, device="cpu", **kwargs):

    x = torch.randn((num_images, *img_shape), device=device)
    model.eval()

    vis_steps = kwargs.get("vis_steps", [99, 80, 60, 40, 20, 10, 5, 1])
    vis_images = []

    if kwargs.get("generate_video", False):
        outs = []

    for time_step in tqdm(reversed(range(1, timesteps)),
                          total=timesteps-1, desc="Sampling :: "):

        ts = torch.ones(num_images, dtype=torch.long, device=device) * time_step
        z = torch.randn_like(x) if time_step > 1 else torch.zeros_like(x)

        predicted_noise = model(x, ts)

        beta_t = get(sd.beta, ts)
        one_by_sqrt_alpha_t = get(sd.one_by_sqrt_alpha, ts)
        sqrt_one_minus_alpha_cumulative_t = get(sd.sqrt_one_minus_alpha_cumulative, ts)

        x = (
            one_by_sqrt_alpha_t
            * (x - (beta_t / sqrt_one_minus_alpha_cumulative_t) * predicted_noise)
            + torch.sqrt(beta_t) * z
        )

        if time_step in vis_steps:
            img = inverse_transform(x).type(torch.uint8)
            grid = torchvision.utils.make_grid(img, nrow=nrow, pad_value=255.0).cpu()
            vis_images.append((time_step, grid))

        if kwargs.get("generate_video", False):
            x_inv = inverse_transform(x).type(torch.uint8)
            grid = torchvision.utils.make_grid(x_inv, nrow=nrow, pad_value=255.0).cpu()
            ndarr = torch.permute(grid, (1, 2, 0)).numpy()[::-1, :, ::-1]
            outs.append(ndarr)

    if kwargs.get("show_process", True):
        fig, ax = plt.subplots(1, len(vis_images), figsize=(15, 3))

        for i, (t, grid) in enumerate(vis_images):
            ax[i].imshow(grid.permute(1, 2, 0))
            ax[i].set_title(f"t={t}", fontsize=8)
            ax[i].axis("off")

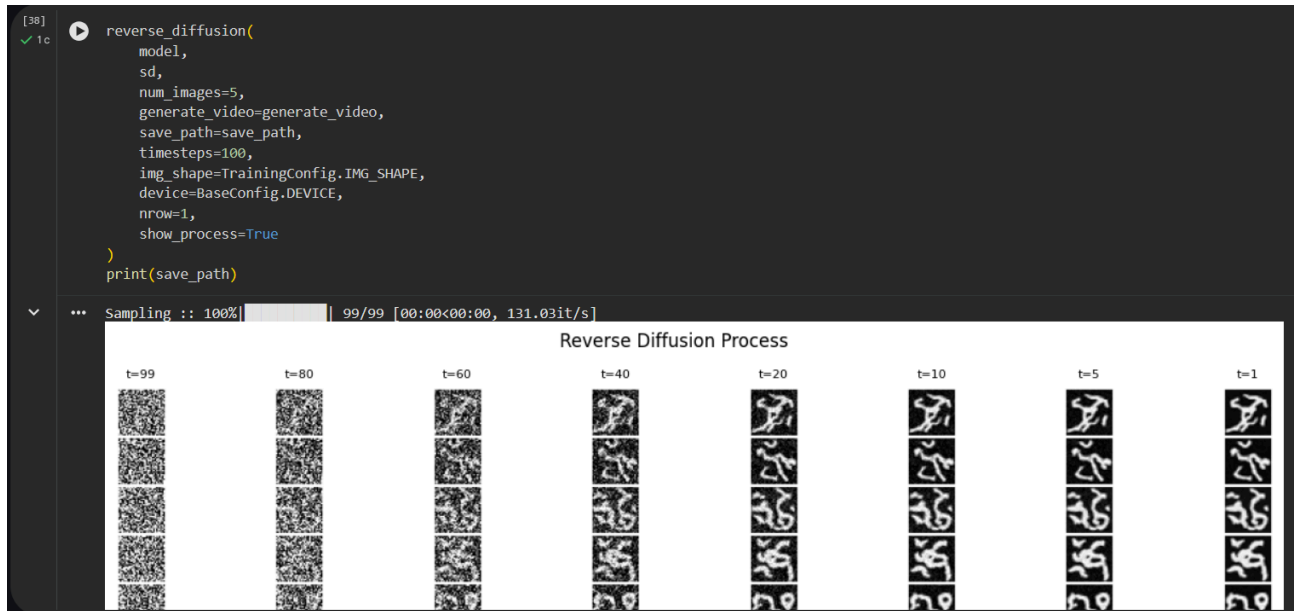
        plt.suptitle("Reverse Diffusion Process", y=1.05)
        plt.show()

    if kwargs.get("generate_video", False):
        frames2vid(outs, kwargs['save_path'])
        display(Image.fromarray(outs[-1][::-1, :, ::-1]))
    return None

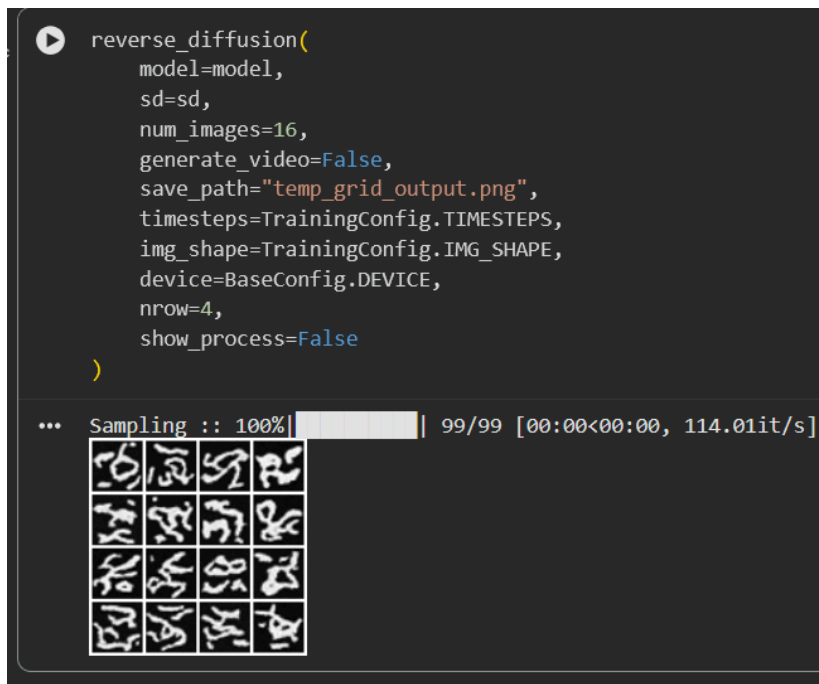
```


else:

```
x = inverse_transform(x).type(torch.uint8)
grid = torchvision.utils.make_grid(x, nrow=nrow, pad_value=255.0).cpu()
pil_image = TF.functional.to_pil_image(grid)
pil_image.save(kwargs['save_path'], format=kwargs['save_path'][-3:].upper())
display(pil_image)
return None
```



Додати grid зі згенерованих 16 зображень



GitHub:

https://github.com/NureVovkDmytro/LLM_LW_2026/blob/main/Generating_MNIST_using_DDPMs.ipynb

Висновок

Під час виконання лабораторної роботи було практично опановано принципи реалізації та використання базової дифузійної моделі DDPM (Denoising Diffusion Probabilistic Models) на основі бібліотеки PyTorch. У ході дослідження було успішно адаптовано архітектуру нейромережі UNet під набір даних MNIST із розміром зображень 32×32 пікселі. Кількість рівнів UNet було зменшено до 3, а також встановлено один резидуальний блок на рівень. Налаштування 100 часових кроків для процесів прямої та зворотної дифузії разом із навчанням моделі протягом 5 епох дозволили досягти стабільного зниження функції втрат.

Крім того, було реалізовано графічний аналіз етапів спотворення та відновлення даних. Візуалізація forward diffusion підтвердила правильність алгоритму поступового накладання гаусового шуму на вхідні цифри. Оновлена функція reverse_diffusion наочно продемонструвала здатність мережі крок за кроком генерувати чіткі структури з випадкового шуму. Фінальне виведення сітки з 16 згенерованих зображень підтвердило ефективність налаштованої моделі та успішне засвоєння практичних навичок роботи з генеративними нейромережами.